Assignment 6 Visitor Pattern

Software Design Patterns | Advanced File System | Java 17 and Maven

Amirkhan Tulkibayev
Group SE-2522

https://github.com/hann-qaz/Assignment6_visitor_pattern

Introduction and purpose

This project models a small file system in memory. It uses the Visitor pattern to calculate total size, search for files and collect statistics over the same tree. These operations are separate from the file and directory classes, so another operation can be added without editing the elements.

Structure and double dispatch

Element declares accept(Visitor). Visitor declares visit(FileNode) and visit(DirectoryNode). FileNode stores a name and a non-negative byte size. DirectoryNode holds child nodes, allowing nested and empty directories. Their shared Node class stores an immutable name and a parent reference.

```
Element element = new FileNode("notes.txt", 120);
TotalSizeVisitor visitor = new TotalSizeVisitor();
visitor.scan(element);
```

Inside scan, root.accept(this) selects FileNode.accept at runtime. That method calls visitor.visit(this); because this is a FileNode, Java selects the visit(FileNode) overload and runs the visitor implementation. DirectoryNode.accept selects visit(DirectoryNode) in the same way. This is double dispatch.

Operations and traversal

TotalSizeVisitor adds each file size once and gives directories no byte size. SearchVisitor combines a configurable filename predicate with size >= minimumSize. StatisticsVisitor counts files and directories, including a directory root, and finds maximum depth.

Visitors own the recursive traversal: visit(DirectoryNode) calls child.accept(this) once for each child. The accept methods only dispatch. Search paths omit the chosen directory root, use / and are sorted lexicographically. A file root uses its filename. Depth counts edges from the selected root at depth 0, including empty directories.

Integrity and reusable state

DirectoryNode.addChild rejects duplicate sibling names, multiple parents and cycles before changing the tree. FileNode rejects negative sizes; Node rejects invalid names. getChildren returns an immutable copy. Math.addExact rejects overflow when adding file sizes.

Each scan(root) resets the visitor results before traversal. Search also clears its path; statistics resets its current depth. The same instance can be reused sequentially without multiplying old results. Direct accept calls are individual traversal steps; results are read after a successful scan.

Five concrete Clean Code examples

1 Meaningful naming

StatisticsVisitor.java, lines 6-8

```
private int fileCount;
private int directoryCount;
private int maximumDepth;
```

The names fileCount, directoryCount and maximumDepth directly describe the three measured values, so the reader does not need to guess their meaning.

2 Single Responsibility

TotalSizeVisitor.java, lines 14-16

```
public void visit(FileNode file) {
    totalSize = Math.addExact(totalSize, file.getSize());
}
```

File size calculation belongs to TotalSizeVisitor; FileNode stores file data and DirectoryNode maintains the tree, so changing this calculation affects one visitor.

3 Small methods

FileNode.java, lines 14-16

```
public long getSize() {
    return size;
}
```

getSize does one simple task and returns the stored byte size, making it easy to read and check without unrelated logic.

4 Encapsulation

DirectoryNode.java, lines 34-36

```
public List<Node> getChildren() {
    return List.copyOf(children);
}
```

List.copyOf prevents callers from editing the child collection, so adding a node must go through addChild and its integrity checks.

5 Elimination of type checking chains

FileNode.java, lines 19-21

```
public void accept(Visitor visitor) {
    visitor.visit(this);
}
```

visitor.visit(this) selects visit(FileNode) through the Visitor contract; DirectoryNode.accept selects its own overload in the same way, with no instanceof, casts or reflection.

Program data and sample output

Main.createDemoTree() builds workspace with notes.txt (120 bytes), documents/notes.txt (340), documents/plan.pdf (800), documents/archive/old.txt (60), images/logo.png (2048), and an empty directory. The default search is case-sensitive and requires a name containing .txt and at least 120 bytes.

```
Root: workspace
Total size: 3368 bytes
Search: name contains '.txt', size >= 120
  documents/notes.txt
  notes.txt
Files: 5
Directories: 5
Maximum depth: 3
```

The sum is 120 + 340 + 800 + 60 + 2048 = 3368 bytes. The directory count includes workspace, documents, archive, images and empty. The deepest file is documents/archive/old.txt at depth 3. notes.txt is included because the 120-byte threshold is inclusive.

Automated test summary

mvn test was run on JDK 17.0.21 with Maven 3.9.11. Result: 25 tests passed, 0 failures, 0 errors and 0 skipped. Maven reported BUILD SUCCESS.

Test class	Tests	Verified behavior
DoubleDispatchTest	2	Correct overloads; accept does not walk
VisitorsTest	10	Nested size, statistics, repeat scans, order
SearchVisitorTest	7	Both filters, sorted paths, no matches
TreeIntegrityTest	6	Ownership, cycles, names, immutable children

Traversal and extension trade offs

Visitor-owned traversal keeps this small project easy to follow, but repeats the child loop in each visitor. A shared walker would centralize that loop and need additional traversal context for paths and depth. Recursive traversal uses stack space proportional to tree depth.

StatisticsVisitor was added without changing FileNode, DirectoryNode or their accept methods. This shows the extension boundary: a new operation adds a visitor. A new element type, such as a symbolic link, would require another visit overload in Visitor and updates to every existing visitor.

UML class diagram

Element and Visitor define the contracts. FileNode and DirectoryNode are the concrete elements. TotalSizeVisitor, SearchVisitor and StatisticsVisitor implement the operations. DirectoryNode owns its children; each child has at most one parent.

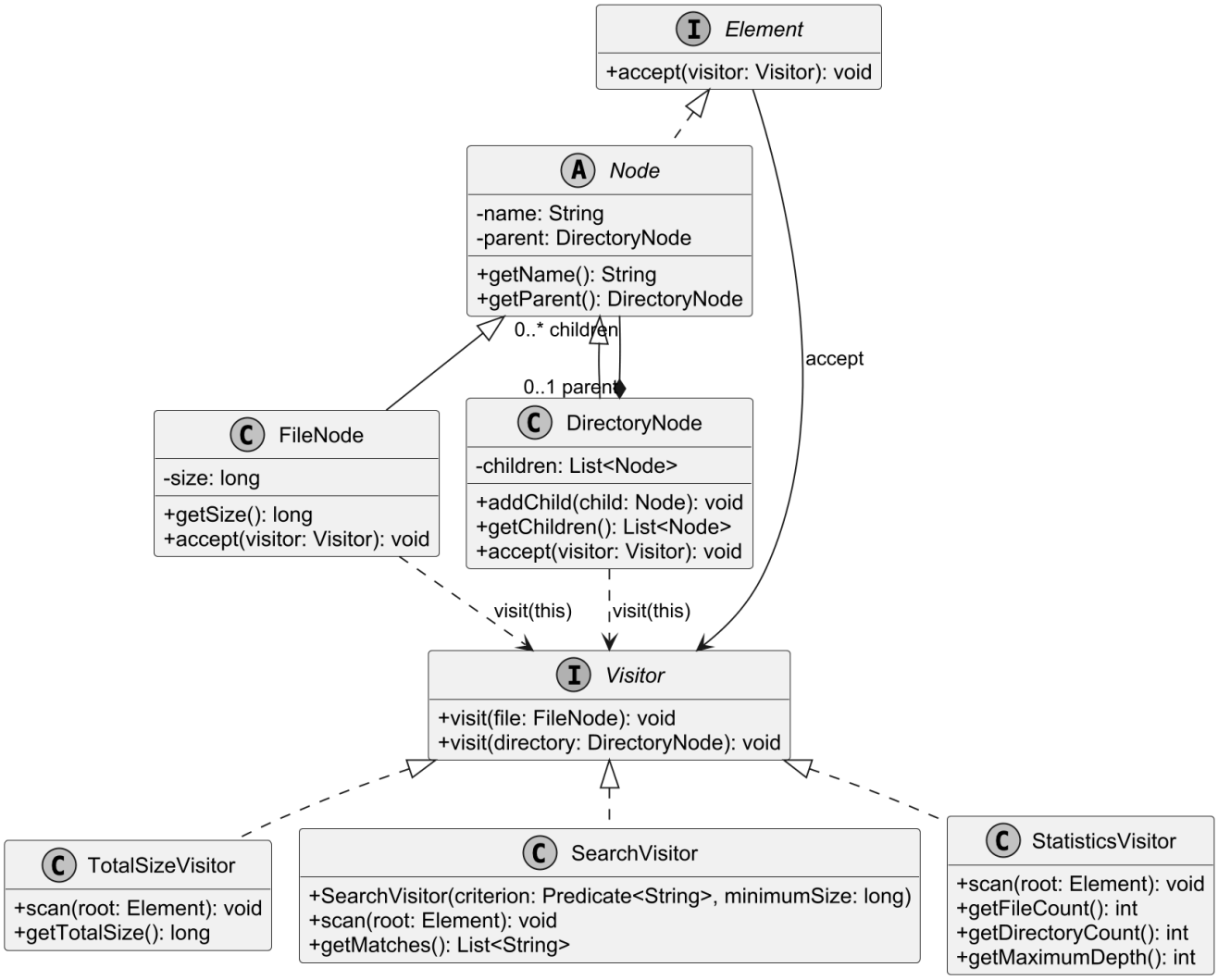


Figure 1. The file system structure and all three visitors. Each concrete visitor implements the two visit overloads declared by Visitor; these methods are omitted from the concrete visitor boxes to keep the diagram compact.

The editable PlantUML class and sequence diagrams and their SVG versions are included in docs.